

Laporan Tugas Kecil 3 IF2211

Strategi Algoritma Ice Sliding

Puzzle Solver



Penulis:

Muhammad Faiz Alfikrona
18223009

Muhamad Hasbullah Faris
18223014

Institut Teknologi Bandung

Pendahuluan

Program ini menyelesaikan permainan ice sliding puzzle. Pemain dimulai dari tile Z dan harus mencapai tile goal O. Namun sebelum goal dianggap valid, pemain harus mengunjungi checkpoint bernomor 0, 1, 2, dan seterusnya secara berurutan. Jika pemain menyentuh checkpoint dengan urutan yang salah, gerakan tersebut dianggap tidak valid.

Perbedaan penting puzzle ini dibanding grid pathfinding biasa adalah cara Bergeraknya. Pemain tidak bergerak satu sel per aksi. Ketika memilih arah atas, bawah, kiri, atau kanan, pemain akan terus meluncur sampai berhenti karena menabrak obstacle/dinding, mencapai goal, atau gerakannya menjadi tidak valid. Karena itu, satu edge pada graf pencarian merepresentasikan satu aksi slide penuh, bukan perpindahan satu petak.

Representasi Tile

Simbol	Arti	Keterangan
Z	Start	Posisi awal pemain.
O	Goal	Tujuan akhir yang hanya valid jika semua checkpoint sudah dikunjungi.
*	Path	Tile biasa yang dapat dilewati.
X	Obstacle	Pemain berhenti sebelum obstacle.
L	Lava	Jika slide menyentuh lava, gerakan tidak valid.
0-9	Checkpoint	Harus dikunjungi berurutan dari angka terkecil.

Dasar Teori dan Rancangan Algoritma

Model Pencarian

Graf Implisit dan State

Puzzle dimodelkan sebagai graf implisit. Node graf tidak dibuat semuanya di awal, melainkan dibangkitkan saat algoritma mengeksplorasi state. Sebuah state terdiri dari dua komponen:

- Posisi pemain, yaitu pasangan baris dan kolom.
- Visited checkpoint bitmask, yaitu bit yang menandai checkpoint mana saja yang sudah dikunjungi.

Dua state dengan posisi sama belum tentu identik jika checkpoint yang sudah dikunjungi berbeda. Contohnya, berada di koordinat yang sama sebelum mengunjungi checkpoint 2 dan sesudah mengunjungi checkpoint 2 adalah kondisi pencarian yang berbeda. Karena itu, state program menggunakan kombinasi Position dan Visited.

Transisi state dibentuk dengan mencoba empat arah. Fungsi slide berjalan sel demi sel sampai kondisi berhenti ditemukan. Biaya satu transisi adalah jumlah cost tile yang dilewati selama slide tersebut. Jika slide menyentuh lava, keluar papan, atau mengambil checkpoint yang belum boleh dikunjungi, transisi dibuang.

Transisi Slide dan Cost

Satu aksi pada puzzle adalah satu slide penuh. Program tidak membuat edge untuk setiap perpindahan satu sel; program hanya membuat edge dari state sebelum slide ke state tempat pemain akhirnya berhenti. Konsekuensinya, branching factor maksimum adalah empat karena dari satu state hanya ada empat arah slide yang dicoba.

Cost slide dihitung dari akumulasi cost tile yang dilewati. Obstacle berfungsi sebagai penghenti slide, lava membuat slide tidak valid, dan checkpoint hanya boleh diambil jika nomornya adalah checkpoint berikutnya yang wajib dikunjungi. Goal hanya diterima sebagai solusi jika seluruh checkpoint sudah ada di bitmask state.

Heuristik

Definisi Umum Heuristik

Heuristik adalah fungsi estimasi yang menilai seberapa jauh sebuah state dari penyelesaian. Dalam program ini, heuristik menerima state dan board, lalu mengembalikan bilangan bulat. Nilai yang lebih kecil berarti state dianggap lebih dekat ke tujuan.

Target Berikutnya dan Chain Checkpoint

Karena checkpoint harus dikunjungi berurutan, estimasi tidak cukup hanya melihat goal. Program memiliki konsep next target, yaitu checkpoint pertama yang belum dikunjungi. Jika semua checkpoint sudah dikunjungi, target berikutnya adalah goal. Untuk heuristik yang lebih panjang, program membentuk

chain dari posisi sekarang, checkpoint yang belum dikunjungi, sampai akhirnya ke goal. Chain ini digunakan untuk menghitung estimasi jarak melalui checkpoint yang masih tersisa.

Heuristik 1: Manhattan ke Target Berikutnya

$$H1(s) = \text{Manhattan}(\text{posisi sekarang}, \text{checkpoint berikutnya atau goal})$$

H1 adalah heuristik paling sederhana. Ia hanya melihat target langsung berikutnya, tidak memperhitungkan checkpoint setelahnya. Kelebihannya adalah murah dihitung dan sering cukup baik untuk mengarahkan pencarian pada puzzle kecil. Kekurangannya, H1 bisa terlalu lokal: state yang dekat ke checkpoint berikutnya belum tentu baik untuk checkpoint berikutnya lagi.

Pada grid biasa, Manhattan sering menjadi lower bound jarak. Pada puzzle sliding, satu aksi slide dapat melewati banyak petak, tetapi cost dihitung dari tile yang dilewati. Jika semua biaya tile positif, Manhattan ke target berikutnya masih dapat dipakai sebagai estimasi jarak geometris dasar.

Heuristik 2: Chain Manhattan

$$H2(s) = \text{Manhattan}(\text{posisi}, \text{checkpoint berikutnya}) + \dots + \text{Manhattan}(\text{checkpoint terakhir}, \text{goal})$$

H2 memperhitungkan seluruh checkpoint yang belum dikunjungi sampai goal. Heuristik ini biasanya lebih informatif daripada H1 karena tidak hanya mengejar target terdekat, tetapi juga memperkirakan beban perjalanan total yang masih harus ditempuh.

Contohnya adalah jika checkpoint berikutnya dekat, tetapi setelah itu goal sangat jauh, H1 akan memberi nilai kecil, sedangkan H2 memberi nilai lebih besar karena memperhitungkan sisa rute setelah checkpoint tersebut.

Heuristik 3: Chain Manhattan dengan Penalti Checkpoint Tersisa

$$H3(s) = H2(s) + \text{jumlah checkpoint belum dikunjungi}$$

H3 menambahkan penalti untuk setiap checkpoint yang belum dikunjungi. Tujuannya adalah mendorong pencarian agar menyukai state yang sudah menyelesaikan lebih banyak checkpoint. Namun karena penalti tambahan ini tidak selalu merepresentasikan biaya minimum yang benar-benar harus dibayar, H3 tidak dijamin admissible.

Heuristik 4: Chain Manhattan dikalikan Biaya Tile Minimum

$$H4(s) = H2(s) * \text{minCostTile}$$

H4 mencoba mengubah estimasi jarak geometris menjadi estimasi biaya. Jika setiap petak yang dapat dilewati memiliki cost minimal minCostTile , maka perjalanan sejauh sejumlah petak secara kasar membutuhkan biaya setidaknya jarak tersebut dikali biaya minimum.

H4 dapat lebih relevan daripada H2 pada board yang memiliki variasi cost tile besar, karena H2 hanya menghitung jarak dan tidak melihat cost.

Heuristik 5: Heuristik 4 dengan Estimasi Biaya Slide

$$H5(s) = H4(s) + \text{estimasiSlideCost}(s)$$

H5 menambahkan estimasi tambahan berdasarkan rata-rata panjang slide pada board. Ide dasarnya adalah puzzle sliding memiliki struktur gerak berbeda dari grid biasa: pemain tidak memilih satu langkah kecil, tetapi satu slide penuh. H5 mencoba memperkirakan tambahan biaya karena karakteristik slide.

Karena estimasi tambahan ini dapat melebihi biaya aktual terbaik pada beberapa konfigurasi, H5 tidak dijamin admissible. H5 lebih cocok untuk GBFS atau eksperimen ketika kecepatan lebih penting daripada jaminan optimalitas.

Ringkasan Heuristik

Heuristik	Definisi	Admissible
1	Manhattan ke Target Berikutnya	Ya, dengan asumsi lower bound geometris valid
2	Total Manhattan sepanjang Chain Checkpoint	Ya, dengan asumsi lower bound geometris valid
3	Chain Manhattan dengan Penalti Checkpoint Tersisa	Tidak dijamin
4	Chain Manhattan dikalikan Biaya Tile Minimum	Ya, jika biaya minimum adalah lower bound valid
5	Heuristik 4 dengan Estimasi Biaya Slide	Tidak dijamin

Algoritma Pathfinding

Uniform Cost Search (UCS)

UCS memilih node dengan total biaya aktual terkecil dari start. Pada implementasi ini, setiap node menyimpan G, yaitu cost dari start sampai node tersebut. Priority queue mengurutkan node berdasarkan nilai F. Untuk UCS, F diisi sama dengan G. Langkah UCS dalam program:

1. Buat node awal dengan posisi start, checkpoint kosong, cost $G = 0$.
2. Masukkan node awal ke priority queue.
3. Ambil node dengan cost terkecil.
4. Jika node sudah berada di goal dan semua checkpoint telah dikunjungi, solusi ditemukan.
5. Jika belum, coba slide ke empat arah.
6. Untuk setiap slide valid, buat node anak dengan G baru.
7. Masukkan node anak ke priority queue jika belum ada state sama dengan cost lebih baik.

UCS optimal selama semua biaya edge tidak negatif. Pada puzzle ini biaya tile bernilai positif untuk tile yang dapat dilewati, sehingga UCS dapat digunakan sebagai pembanding solusi optimal.

Greedy Best First Search (GBFS)

GBFS memilih node berdasarkan nilai heuristik saja. Artinya, program memperkirakan state mana yang terlihat paling dekat ke tujuan berikutnya, lalu mengeksplorasi state itu terlebih dahulu. Pada GBFS, F diisi dengan H, bukan $G + H$.

Langkah GBFS dalam program:

1. Hitung heuristik state awal.
2. Masukkan state awal ke priority queue dengan prioritas H.
3. Ambil state dengan heuristik terkecil.
4. Jika goal valid tercapai, kembalikan solusi.
5. Jika belum, hasilkan semua slide valid dan hitung heuristiknya.
6. Masukkan node anak dengan prioritas heuristik.

GBFS dapat sangat cepat bila heuristik mengarah dengan baik, tetapi tidak menjamin cost optimal karena tidak memperhitungkan biaya yang sudah dibayar untuk mencapai state.

A*

A* menggabungkan keunggulan UCS dan pencarian berbasis heuristik. A* tetap memperhatikan cost aktual dari start, tetapi juga memakai estimasi sisa jarak ke tujuan. Pada implementasi ini, prioritas node adalah:

$$f(n) = g(n) + h(n)$$

Node dengan nilai f paling kecil dianggap paling menjanjikan karena memiliki estimasi total biaya solusi terendah.

Langkah A* dalam program:

1. Buat node awal dengan $G = 0$.
2. Hitung H menggunakan heuristik yang dipilih.
3. Set $F = G + H$ dan masukkan ke priority queue.
4. Ambil node dengan F terkecil.
5. Jika goal valid tercapai, rekonstruksi path dari parent pointer.
6. Jika belum, bangkitkan semua slide valid.
7. Untuk tiap anak, hitung G, H, dan F.
8. Gunakan map best cost untuk menghindari ekspansi state yang sama dengan cost lebih buruk.

Jika heuristik admissible dan konsisten, A* dapat menemukan solusi optimal dengan ekspansi yang biasanya lebih sedikit daripada UCS.

IDA*

IDA* adalah variasi A* yang memakai pendekatan iterative deepening. Alih-alih menyimpan frontier besar dalam priority queue, IDA* melakukan DFS dengan batas nilai f. Jika suatu node memiliki $f(n)$ lebih besar daripada batas saat ini, cabang dipotong.

Langkah IDA* dalam program:

1. Hitung f node awal dan jadikan sebagai batas awal.
2. Lakukan DFS dari start.
3. Jika $f(n) > \text{bound}$, hentikan cabang dan catat nilai f tersebut sebagai kandidat bound berikutnya.
4. Jika goal valid ditemukan, solusi dikembalikan.
5. Jika belum ditemukan, ulangi DFS dengan bound terkecil yang sebelumnya terpotong.

IDA* menggunakan memori lebih kecil daripada A*, tetapi dapat memperluas state yang sama berkali-kali pada iterasi bound yang berbeda.

Analisis $f(n)$, $g(n)$, Optimalitas, dan Kompleksitas

$g(n)$ adalah total cost aktual dari start ke node n . Pada program, nilai ini bertambah setiap kali slide valid dilakukan. Biaya slide adalah jumlah cost tile yang dilewati selama slide tersebut.

$h(n)$ adalah estimasi sisa cost atau jarak menuju penyelesaian. Nilainya tergantung heuristik yang dipilih.

$f(n)$ adalah nilai prioritas. Definisi per algoritma:

Algoritma	Prioritas	Keterangan
UCS	$f(n) = g(n)$	Pilih cost aktual terkecil
GBFS	$f(n) = h(n)$	Pilih state yang tampak paling dekat ke tujuan
A*	$f(n) = g(n) + h(n)$	Pilih estimasi total cost terkecil
IDA*	$f(n) = g(n) + h(n)$	Iterative Deepening Search dengan pendekatan A*

Implementasi Program

Struktur Source Program

- `src/cmd/cli/main.go`: CLI, output hasil, playback CLI, dan save solusi.
- `src/cmd/gui/main.go` dan `src/cmd/gui/screens.go`: GUI, pemilihan algoritma, pemilihan heuristik, playback, dan save TXT.
- `src/internal/algorithm`: UCS, GBFS, A*, IDA*, priority queue, dan struktur Result.
- `src/internal/board`: parser input, state, arah, dan aturan slide.
- `src/internal/heuristic`: H1 sampai H5.

Implementasi Kode Algoritma dan Heuristik

Struktur Algoritma di `algorithm.go`

Semua algoritma mengikuti interface yang sama:

```
type Algorithm interface {  
    Search(b *board.Board, start board.State, h heuristic.Func) Result
```



```
}
```

Dengan interface ini, CLI dan GUI tidak perlu tahu detail UCS, GBFS, A*, atau IDA*. Keduanya cukup menyimpan nilai bertipe `algorithm.Algorithm`, lalu memanggil `Search`.

`Struct Node` menyimpan semua informasi yang dibutuhkan saat pencarian:

- `State`: posisi dan checkpoint bitmask.
- `G`: cost aktual dari start.
- `H`: nilai heuristik.
- `F`: nilai prioritas.
- `Parent`: pointer ke node sebelumnya untuk rekonstruksi solusi.
- `Move`: arah slide yang menghasilkan node ini.

Ketika solusi ditemukan, fungsi `reconstructPath` berjalan mundur dari node goal ke parent-parent sebelumnya. Arah gerakan dan state disisipkan ke depan slice sehingga hasil akhirnya berurutan dari start sampai goal. Fungsi `allVisited` mengecek apakah semua checkpoint pada board sudah ada di bitmask state.

Priority Queue di `heap.go`

UCS, GBFS, dan A* memakai priority queue yang dibungkus dalam `PriorityQueue`. Implementasinya menggunakan package standar `container/heap`. Setiap item queue membungkus pointer ke `Node`.

Fungsi `Less` membandingkan nilai `node.F`. Jika dua node memiliki nilai `F` sama, queue memakai urutan masuk (`order`) sebagai tie breaker. Ini membuat hasil lebih deterministik: node yang masuk lebih dulu akan diproses lebih dulu ketika prioritasnya sama.

Alur penggunaan queue:

1. `NewPriorityQueue` membuat heap kosong.
2. `Push` memasukkan node baru bersama nomor urutan.
3. `Pop` mengambil node dengan nilai `F` terkecil.
4. `Len` dipakai loop pencarian untuk mengetahui apakah frontier masih berisi node.

Kode UCS di `ucs.go`

UCS dimulai dengan mencatat waktu awal memakai `time.Now()`. Setelah itu program membuat priority queue, map best, dan counter iterasi.

```
pq := NewPriorityQueue()
best := map[board.State]int{}
pq.Push(&Node{State: start, G: 0, F: 0})
```

Map best menyimpan cost terbaik yang pernah ditemukan untuk suatu state. Ketika node diambil dari queue, program mengecek:

```
if prev, seen := best[cur.State]; seen && prev <= cur.G {
    continue
}
```

Jika state yang sama sudah pernah dicapai dengan cost lebih kecil atau sama, node saat ini tidak perlu diproses lagi. Ini menghindari ekspansi berulang yang tidak berguna.

Setelah itu, UCS mengecek goal. Jika posisi sudah di goal dan semua checkpoint sudah dikunjungi, path direkonstruksi dan dikembalikan sebagai Result. Jika belum goal, program mencoba empat arah:

```
for _, dir := range []board.Direction{board.Up, board.Down, board.Left,
board.Right} {
    ns, cost, valid := board.Slide(b, cur.State, dir)
    ...
}
```

Jika slide valid, cost baru dihitung sebagai $g := cur.G + cost$. Untuk UCS, node anak dimasukkan dengan $F = g$, sehingga priority queue selalu memilih total cost aktual terkecil.

Kode GBFS di gbfs.go

GBFS mirip dengan UCS dalam struktur loop, tetapi prioritasnya berbeda. Pada awal pencarian, program menghitung heuristik start:

```
hVal := 0
if h != nil {
    hVal = h(start, b)
}
pq.Push(&Node{State: start, G: 0, H: hVal, F: hVal})
```

Perhatikan bahwa F diisi dengan hVal, bukan $G + H$. Ini membuat GBFS hanya melihat estimasi ke depan.

GBFS memakai map visited, bukan map best cost. Begitu state pernah diproses, state itu tidak diproses lagi. Ketika membangkitkan node anak, program tetap menghitung G agar total cost solusi bisa dilaporkan, tetapi prioritas queue tetap hanya heuristik:

```
pq.Push(&Node{
    State: ns,
    G: cur.G + cost,
    H: hVal,
    F: hVal,
    Parent: cur,
    Move: dir,
})
```

Karena G tidak mempengaruhi prioritas, GBFS dapat menemukan solusi lebih cepat, tetapi solusi tersebut tidak dijamin memiliki cost minimum.

Kode A* di astar.go

A* menggunakan struktur yang sangat mirip UCS, tetapi setiap node memiliki nilai heuristik. Pada start:

```
pq.Push(&Node{State: start, G: 0, H: hVal, F: hVal})
```

Karena $G = 0$, nilai F awal sama dengan H. Ketika membuat anak:

```
g := cur.G + cost
hVal := h(ns, b)
pq.Push(&Node{
    State: ns,
    G: g,
    H: hVal,
    F: g + hVal,
    Parent: cur,
    Move: dir,
})
```

Inilah bagian paling penting dari A*: prioritas tidak hanya melihat jarak perkiraan ke goal, tetapi juga biaya aktual yang sudah dibayar. A* juga memakai map best seperti UCS agar state yang sama dengan cost lebih buruk tidak diekspansi.

Kode IDA* di idastar.go

IDA* memiliki dua fungsi utama: method Search dan fungsi rekursif idaSearch. Pada awal pencarian, program menghitung nilai F root, lalu menjadikannya bound awal:

```
root := &Node{State: start, G: 0, H: hVal, F: hVal}
bound := root.F
```

Setelah itu, program melakukan loop. Pada setiap loop, dibuat map path untuk mencegah siklus dalam jalur DFS saat ini. Fungsi idaSearch mengembalikan dua hal: bound berikutnya dan node solusi jika ditemukan.

```
nextBound, found := idaSearch(b, root, bound, h, path, &iterations)
```

Di dalam idaSearch, program pertama-tama menaikkan counter iterasi. Lalu:

```
if cur.F > bound {
    return cur.F, nil
}
```

Jika nilai F melebihi bound, cabang dipotong. Nilai F tersebut dikembalikan sebagai kandidat bound berikutnya. Jika node adalah goal valid, node dikembalikan sebagai solusi. Jika belum, fungsi membangkitkan semua slide valid dan memanggil dirinya sendiri secara rekursif.

Jika semua cabang gagal, bound dinaikkan ke nilai minimum dari semua cabang yang terpotong. Jika tidak ada bound baru, berarti solusi tidak ditemukan.

Kode Heuristik di funcs.go

File heuristik berisi fungsi bantu dan definisi H1 sampai H5. Fungsi paling dasar adalah abs dan manhattan:

```
func manhattan(a, b board.Position) int {
    return abs(a.Row-b.Row) + abs(a.Col-b.Col)
}
```

Manhattan menghitung jarak grid tanpa memperhatikan obstacle. Nilai ini cepat dihitung dan menjadi dasar H1, H2, H3, dan H4.

nextTarget

```
func nextTarget(s board.State, b *board.Board) board.Position {
```

```

for i, cp := range b.Checkpoints {
    if !s.HasVisited(i) {
        return cp
    }
}
return b.Goal
}

```

Fungsi ini mencari checkpoint pertama yang belum dikunjungi. Jika semuanya sudah dikunjungi, fungsi mengembalikan goal. H1 memakai fungsi ini untuk menentukan target langsung berikutnya.

remainingChain dan chainManhattan

remainingChain membuat daftar posisi yang harus dilalui dari state saat ini sampai goal. Daftarnya dimulai dari posisi pemain, lalu checkpoint yang belum dikunjungi, lalu goal. chainManhattan menjumlahkan Manhattan antar pasangan berurutan dalam chain tersebut.

```

chain := []board.Position{s.Pos}
for i, cp := range b.Checkpoints {
    if !s.HasVisited(i) {
        chain = append(chain, cp)
    }
}
chain = append(chain, b.Goal)

```

H2 memakai hasil chainManhattan. Karena checkpoint wajib berurutan, chain ini sesuai dengan struktur objektif puzzle.

minTileCost

minTileCost mencari biaya tile terkecil dari semua tile yang dapat dilewati. Obstacle dan lava diabaikan. Fungsi ini dipakai H4 untuk mengubah estimasi jarak menjadi estimasi biaya minimum.

```

if tile == board.TileObstacle || tile == board.TileLava {
    continue
}

```

slideLengthEstimate dan averageSlideLength

Dua fungsi ini dipakai H5. slideLengthEstimate memperkirakan panjang slide dari suatu posisi dan arah tanpa benar-benar mengubah state. Jika slide keluar papan

atau menyentuh lava, estimasi dianggap tidak valid. `averageSlideLength` menghitung rata-rata panjang slide valid di seluruh board.

Tujuannya adalah memberi gambaran karakteristik board: apakah board cenderung memiliki slide panjang atau pendek. H5 kemudian memakai informasi ini untuk menambahkan estimasi biaya slide.

Overview Heuristik dalam Kode

Heuristik	Kode
H1	<code>manhattan(s.Pos, nextTarget(s, b))</code>
H2	<code>chainManhattan(s, b)</code>
H3	<code>H2(s, b) + unvisitedCount(s, b)</code>
H4	<code>H2(s, b) * minTileCost(b)</code>
H5	<code>H4(s, b) + estimatedSlideCost(s, b)</code>

File `heuristic.go` juga menyimpan daftar heuristik dalam slice `All`. GUI memakai daftar ini untuk membuat tombol pilihan heuristik secara otomatis, sehingga penambahan heuristik baru dapat dilakukan dengan menambah entry di daftar tersebut.

Cara Menjalankan Program

Program membutuhkan Go dan dependency yang tercatat pada `src/go.mod`. Output executable diarahkan ke folder `bin`.

Build executable ke folder `bin`:

```
cd src
go build -o ../bin/cli.exe ./cmd/cli
go build -o ../bin/gui.exe ./cmd/gui
```

Menjalankan CLI tanpa build terpisah:

```
cd src
go run ./cmd/cli
```

Menjalankan GUI tanpa build terpisah:

```
cd src
go run ./cmd/gui
```

Visualisasi dan Evaluasi

Playback dan Visualisasi

CLI Playback

- A: mundur satu step.
- D: maju satu step.
- ESC atau J: lompat ke step tertentu.
- Q: keluar playback.

GUI Playback

- Tombol Prev dan Next untuk mundur/maju.
- Tombol Play/Pause untuk menjalankan playback otomatis.
- Tombol + dan - untuk mengatur kecepatan dalam satuan steps/s.
- Jika playback berada di akhir lalu tombol Play ditekan, playback dimulai ulang dari awal.

File Sampel

Sampel	Isi
sample.txt	7 7 XXXXXXXX X0****X X**X**X X****OX X***1LX XZ**X*X XXXXXXXX 999 999 999 999 999 999 999 999 3 5 2 8 1 999 999 7 4 999 6 9 999 999 2 8 3 5 4 999 999 6 1 7 2 999 999 999 9 3 4 999 8 999 999 999 999 999 999 999 999
sample2.txt	7 7 XXXXXXXX X0**X*X X**X**X X*OX**X X**1*LX XZ****X

Sampel	Isi
	XXXXXXXX 999 999 999 999 999 999 999 999 2 7 5 999 3 999 999 6 4 999 8 1 999 999 5 999 2 6 4 999 999 3 9 7 5 2 999 999 4 1 6 8 3 999 999 999 999 999 999 999 999
sample3.txt	7 7 XXXXXXXX X0***OX X*XXX*X X*XXX*X X*1***X XZXXXXX XXXXXXXX 999 999 999 999 999 999 999 999 2 3 4 5 6 999 999 3 999 999 999 4 999 999 4 999 999 999 3 999 999 5 1 2 3 4 999 999 1 999 999 999 999 999 999 999 999 999 999 999 999
sample4.txt	8 8 XXXXXXXXXX X0****OX X*XXX**X X**XXLXX X*2****X X*XXX**X XZ1****X XXXXXXXXXX 999 999 999 999 999 999 999 999 999 2 3 4 5 6 7 999 999 3 999 999 999 4 5 999 999 4 7 999 999 999 6 999 999 5 2 3 4 5 6 999 999 6 999 999 999 4 3 999 999 1 2 3 4 5 6 999 999 999 999 999 999 999 999 999
sample5.txt	25 25 XXXXXXXXXXXXXXXXXXXXXXXXXXXXX XXXXXXXXXXXXXXXXXXXXXXXXXXXXX XXXXXXXXXXXXXXXXXXXXXXXXXXXXX XXXXXXXXXXXXXXXXXXXXXXXXXXXXX

Sampel	Isi
	999 999 999 999 999 999 999 999 999 999 999 999 999 999 999 4 8 3 7 2 6 1 5 9 4 8 3 7 2 6 1 999 999 999 999 999 999 999 999 999 999 999 999 999 999 4 999 999 999 999 999 999 999 999 999 8 999 999 999 999 999 999 999 999 5 999 999 999 999 999 2 999 999 999 999 999 999 999 999 999 6 999 999 999 999 999 999 999 999 3 999 999 999 999 999 9 999 999 999 999 2 6 1 5 999 4 999 999 999 999 999 999 999 999 1 999 999 999 999 999 999 999 999 999 999 999 999 999 999 999 2 999 999 999 999 999 999 999 999 8 999 999 999 999 999 999 999 999 999 999 999 999 999 999 999 9 999 999 999 999 999 999 999 999 999 999 999 999 999 999 3 7 2 6 1 5 9 4 8 3 7 999 999 999 999 999 1 999 999 999 999 999 999 999 999 999 999 999 999 8 999 999 999 999 999 5 999 999 999 999 999 8 999 999 999 999 999 999 999 999 999 999 999 999 6 999 999 999 999 999 3 999 999 999 999 999 6 999 999 999 999 999 999 999 999 999 999 999 999 4 999 999 999 999 999 1 999 999 999 999 999 4 8 3 7 2 6 1 5 9 4 8 3 999 2 999 999 999 999 999 999 999 999 999 999 999 2 999 999 999 999 999 999 3 999 999 999 999 999 9 999 999 999 7 999 999 999 999 999 999 999 9 999 999 999 999 999 999 1 999 999 999 999 999 7 999 999 999 5 999 999 999 999 999 999 3 7 2 6 1 5 9 4 8 3 7 2 6 1 5 999 999 999 3 999 999 999 999 999 999 999 999 999 999 999 999 999 999 6 999 999 999 999 999 3 999 999 999 1 999 999 999 999 999 4 8 3 7 2 6 1 5 9 4 999 999 999 999 999 999 999 999 999 999 999 999 999 999

Sampel	Isi
	999 999

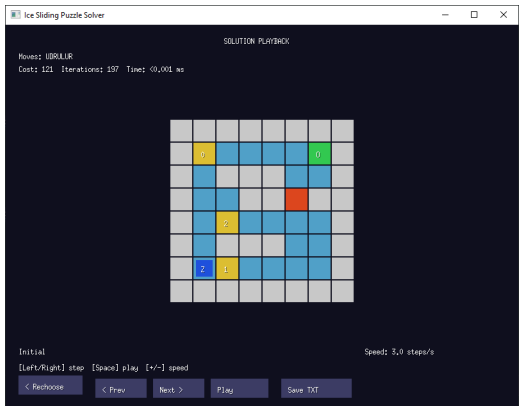
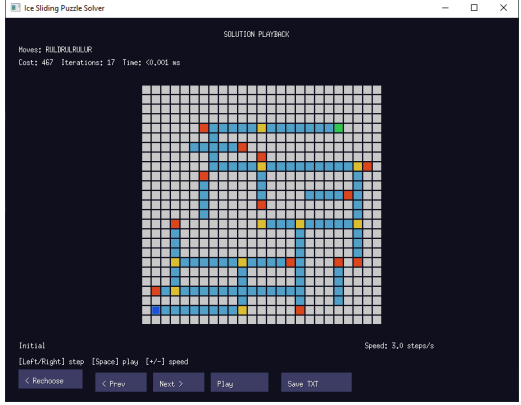
Tangkapan Layar Input dan Output

CLI

Input	Algoritma	Heuristik	Output
../test/sample.txt	UCS	-	Tidak ditemukan solusi.
../test/sample2.txt	GBFS	H2	Tidak ditemukan solusi.
../test/sample3.txt	UCS	UCS	Solusi : URDLRU Cost : 77 Waktu : 0.230 ms Iterasi: 11 Initial XXXXXXXX X0***OX X*XXX*X X*XXX*X X*1***X XZXXXXXX XXXXXXXX Step 1 : U XXXXXXXX XZ***OX X*XXX*X X*XXX*X X*1***X XZXXXXXX XXXXXXXX Step 2 : R XXXXXXXX X****ZX X*XXX*X X*XXX*X X*1***X

Input	Algoritma	Heuristik	Output
			XZXXXXXX XXXXXXX Step 3 : D XXXXXXX X****OX X*XXX*X X*XXX*X X*1**ZX XZXXXXXX XXXXXXX Step 4 : L XXXXXXX X****OX X*XXX*X X*XXX*X XZ****X XZXXXXXX XXXXXXX Step 5 : R XXXXXXX X****OX X*XXX*X X*XXX*X X****ZX XZXXXXXX XXXXXXX Step 6 : U XXXXXXX X****ZX X*XXX*X X*XXX*X X*****X XZXXXXXX XXXXXXX

GUI

Input	Konfigurasi	Output
sample4.txt	IDA* dengan heuristik H1	
sample5.txt	A* dengan heuristik H2	

Analisis Hasil Percobaan

Berikut adalah hasil percobaan yang ditulis dalam format cost / iterasi / waktu. Waktu yang sangat kecil ditampilkan sebagai <0.001 ms karena durasi pencarian lebih kecil daripada resolusi praktis pengukuran milisecond.

Matriks Hasil Lengkap

Kasus	UCS	GBFS H1	GBFS H2	GBFS H3	GBFS H4	GBFS H5
sample3	77 / 11 / <0.001	77 / 8 / <0.001	77 / 8 / <0.001	77 / 8 / <0.001	77 / 8 / <0.001	77 / 8 / <0.001
sample4	121 / 20 / <0.001	121 / 15 / <0.001	121 / 10 / <0.001	121 / 10 / <0.001	121 / 10 / <0.001	121 / 10 / <0.001
sample5	467 / 17 / <0.001	467 / 16 / <0.001	467 / 13 / 0.011	467 / 13 / <0.001	467 / 13 / <0.001	467 / 13 / <0.001

Kasus	A* H1	A* H2	A* H3	A* H4	A* H5
sample3	77 / 11 / <0.001	77 / 11 / 0.020	77 / 11 / <0.001	77 / 11 / 0.010	77 / 11 / <0.001
sample4	121 / 17 / <0.001	121 / 17 / <0.001	121 / 17 / <0.001	121 / 17 / <0.001	121 / 16 / <0.001
sample5	467 / 17 / <0.001	467 / 17 / <0.001	467 / 17 / <0.001	467 / 17 / <0.001	467 / 17 / <0.001

Kasus	IDA* H1	IDA* H2	IDA* H3	IDA* H4	IDA* H5
sample3	77 / 81 / 0.019	77 / 81 / <0.001	77 / 81 / <0.001	77 / 81 / <0.001	77 / 81 / <0.001
sample4	121 / 197 / <0.001	121 / 199 / <0.001	121 / 199 / <0.001	121 / 199 / <0.001	121 / 166 / 0.017
sample5	467 / 171 / <0.001	467 / 178 / <0.001	467 / 170 / <0.001	467 / 178 / <0.001	467 / 179 / 1.030

Solusi yang Dihasilkan

Kasus	Solusi	Cost Optimal Pembanding UCS
sample3	URDLRU	77
sample4	UDRULUR	121
sample5	RULDRULRULUR	467

Analisis Per Algoritma dan Heuristik

Dari hasil lengkap, UCS menjadi baseline karena tidak memakai heuristik dan memilih state berdasarkan cost aktual. Pada tiga sample yang diuji, UCS menghasilkan cost 77, 121, dan 467. Nilai ini digunakan sebagai pembanding cost algoritma lain. Pada pengujian ini, semua kombinasi GBFS, A*, dan IDA* juga menemukan solusi dengan cost yang sama dengan UCS. Walaupun begitu, kesamaan cost pada sample ini tidak mengubah sifat teoretis GBFS bahwa GBFS tetap tidak menjamin optimal karena prioritasnya hanya $h(n)$, bukan $g(n) + h(n)$.

GBFS memiliki jumlah iterasi paling kecil pada semua sample. Pada sample3, semua heuristik GBFS membutuhkan 8 iterasi, lebih kecil daripada UCS yang membutuhkan 11 iterasi. Pada sample4, H2 sampai H5 turun ke 10 iterasi, sedangkan H1 membutuhkan 15 iterasi. Pada sample5, H2 sampai H5

membutuhkan 13 iterasi, sedangkan H1 membutuhkan 16 iterasi. Ini menunjukkan bahwa heuristik chain checkpoint lebih informatif daripada hanya melihat target berikutnya.

A* pada sample ini lebih stabil daripada GBFS. Jumlah iterasi A* dekat dengan UCS karena A* tetap mempertahankan komponen cost aktual $g(n)$. Pada sample4, A* H5 sedikit lebih kecil daripada H1-H4, yaitu 16 iterasi dibanding 17 iterasi. Pada sample5, seluruh heuristik A* menghasilkan 17 iterasi. Hal ini berarti pada board tersebut kombinasi nilai $g(n)$, urutan tie breaker priority queue, dan struktur slide membuat perbedaan antar heuristik tidak banyak mengubah urutan ekspansi A*.

IDA* selalu membutuhkan iterasi lebih banyak daripada UCS, GBFS, dan A* pada sample yang sama. Pada sample4, IDA* H1 membutuhkan 197 iterasi, H2-H4 membutuhkan 199 iterasi, dan H5 turun menjadi 166 iterasi. Pada sample5, IDA* berada pada rentang 170 sampai 179 iterasi. Ini sesuai teori bahwa IDA* hemat memori karena tidak menyimpan frontier besar, tetapi dapat mengulang ekspansi state pada beberapa bound berbeda.

Perbandingan antar heuristik menunjukkan bahwa H2, H3, H4, dan H5 sering menghasilkan iterasi GBFS yang sama pada sample ini. Penyebabnya adalah semua heuristik tersebut memakai struktur chain checkpoint sebagai dasar. H3 menambahkan jumlah checkpoint tersisa, H4 mengalikan dengan cost tile minimum, dan H5 menambahkan estimasi slide, tetapi urutan prioritas antar state yang penting sering tetap sama. H1 lebih lemah karena hanya melihat target berikutnya, sehingga pada sample4 dan sample5 iterasinya lebih besar daripada H2-H5 untuk GBFS.

Waktu eksekusi banyak bernilai <0.001 ms karena input masih kecil untuk komputer modern dan yang diukur hanya waktu pencarian, bukan waktu membaca file, menulis file, atau playback. Karena durasi aktual sangat pendek, iterasi lebih informatif daripada waktu untuk membandingkan perilaku algoritma pada sample ini.

Pada sample5, terdapat jebakan urutan checkpoint seperti pola 5, 4, 6. Algoritma harus mencapai checkpoint 4, lalu bergerak ke kiri untuk mengambil 5, kemudian bergerak ke kanan untuk mengambil 6. Selain itu terdapat lava pada jalur langsung dari checkpoint 8 menuju 9, sehingga solusi harus melakukan detour. Ini menunjukkan bahwa state tidak cukup hanya dinilai dari posisi, tetapi juga harus memperhitungkan checkpoint yang sudah dikunjungi.

Pranala Repositori

https://github.com/hsbu/Tucil3_18223009_18223014

Referensi

- [1] 'Command Documentation - The Go Programming Language'. Accessed: May 10, 2026. [Online]. Available: <https://go.dev/doc/cmd>
- [2] 'About the go command - The Go Programming Language'. Accessed: May 10, 2026. [Online]. Available: https://go.dev/doc/articles/go_command
- [3] 'Install - Ebitengine', Ebitengine - A dead simple 2D game engine for Go. Accessed: May 10, 2026. [Online]. Available: <https://ebitengine.org/en/documents/install.html>
- [4] A. J. Patel, 'Introduction to the A* Algorithm', Red Blob Games, 2014. Accessed: May 10, 2026. [Online]. Available: <https://www.redblobgames.com/pathfinding/a-star/introduction.html>
- [5] A. Patel, 'Heuristics'. Accessed: May 10, 2026. [Online]. Available: <http://theory.stanford.edu/~amitp/GameProgramming/Heuristics.html>
- [6] 'Taxicab geometry', Wikipedia. May 09, 2026. Accessed: May 10, 2026. [Online]. Available: https://en.wikipedia.org/w/index.php?title=Taxicab_geometry&oldid=1353319753
- [7] 'heap package - container/heap - Go Packages'. Accessed: May 10, 2026. [Online]. Available: <https://pkg.go.dev/container/heap>
- [8] 'term package - golang.org/x/term - Go Packages'. Accessed: May 10, 2026. [Online]. Available: <https://pkg.go.dev/golang.org/x/term>
- [9] 'Ebitengine - A dead simple 2D game engine for Go', Ebitengine - A dead simple 2D game engine for Go. Accessed: May 10, 2026. [Online]. Available: <https://ebitengine.org>
- [10] 'A* search algorithm', Wikipedia. May 06, 2026. Accessed: May 10, 2026. [Online]. Available: https://en.wikipedia.org/w/index.php?title=A*_search_algorithm&oldid=1352790550
- [11] 'Iterative deepening A*', Wikipedia. Jan. 30, 2026. Accessed: May 10, 2026. [Online]. Available: https://en.wikipedia.org/w/index.php?title=Iterative_deepening_A*&oldid=1335719951

Pernyataan Penggunaan AI

Tugas ini disusun sepenuhnya tanpa bantuan kecerdasan buatan (*Generative AI*), melainkan hasil pemikiran dan analisis mandiri.



Muhamad Hasbullah F.



Muhammad Faiz A.